

In-Class Python Task Worksheet

Arrays and Algorithm Analysis - Tasks 1 to 3

Student Name: 杨佳宇

Student ID: 20253801028

Date: 2026-09-18

Class/Section: _____

Task 1 - Complexity Challenge

Determine the time complexity of each code fragment and briefly explain why.

A.

```
print(numbers[0])
```

Time Complexity: $O(1)$

Reason: Accessing a single element by index in a Python list is a constant-time operation. The size of the list does not affect the number of steps; only one memory lookup is performed.

B.

```
for i in range(n):
```

```
    print(i)
```

Time Complexity: $O(n)$

Reason: The loop iterates n times, and each iteration performs a constant amount of work (one print). Therefore the total work grows linearly with n .

C.

```
for i in range(n):
```

```
    for j in range(n):
```

```
        print(i, j)
```

Time Complexity: $O(n^2)$

Reason: There are two nested loops, each running n times, so the inner statement executes $n * n = n^2$ times. The total work grows quadratically with n .

D.

```
for i in range(n):
```

```
    print(i)
```

```
for j in range(n):
```

```
    print(j)
```

Time Complexity: $O(n)$

Reason: The two loops are sequential (not nested), so the total iterations are $n + n = 2n$.

Constant factors are dropped in Big-O, so the complexity is $O(n)$.

E.

```
for i in range(n):
```

```
    for j in range(100):
```

```
        print(i, j)
```

Time Complexity: $O(n)$

Reason: The inner loop runs a fixed 100 times regardless of n , so the total iterations are $n * 100 = 100n$. Since 100 is a constant, it is dropped in Big-O, giving $O(n)$.

Task 2 - Logarithmic Complexity Challenge

Study the code below. Predict the printed sequence before running it, then analyze its time complexity.

```
n = 100
```

```
while n > 1:
```

```
    print(n)
```

```
    n = n // 2
```

1. Write the values you expect the program to print:

100, 50, 25, 12, 6, 3

2. What happens to n after every loop iteration?

After each iteration, n is replaced by $n // 2$ (integer division by 2), so n is halved and rounded down to the nearest integer each time, then stopping because n becomes 1

3. State the time complexity in Big-O notation:

Time Complexity: $O(\log n)$

4. Explain why this loop is not $O(n)$:

In an $O(n)$ loop, the number of iterations grows proportionally to n (for example, a loop that decreases n by 1 each time). Here, n is halved each iteration, so the number of iterations grows proportionally to $\log_2(n)$ instead. For $n = 100$, the loop runs about 7 times, far fewer than 100. This logarithmic behavior is the defining characteristic of $O(\log n)$, not $O(n)$.

Extension: If the starting value were 1024 instead of 100, approximately how many times would the loop execute?

Answer: 10 times

Explanation: $1024 = 2^{10}$. Since each iteration halves n , starting from 2^{10} the loop will run until n becomes 1, which takes exactly 10 iterations (1024 -> 512 -> 256 -> 128 -> 64 -> 32 -> 16 -> 8 -> 4 -> 2 -> 1, printing 1024, 512, ..., 2, then stopping because n becomes 1 which is not greater than 1). So the loop body executes 10 times.

Task 3 - Space Complexity

Analyze the auxiliary space complexity of each algorithm. Do not count the input list itself as additional memory.

Algorithm A

```
def calculate_sum(numbers):
```

```
    total = 0
```

```
    for number in numbers:
```

```
        total += number
```

```
    return total
```

A1. Auxiliary Space complexity: $O(1)$

A2. Which variables use additional memory?

Only the variable `total` (and the loop variable `number`) use additional memory. `total` stores a single integer accumulator, and `number` holds one element at a time during iteration. Both occupy a constant amount of memory regardless of the size of the input list, so the auxiliary space is $O(1)$.

Algorithm B

```
def double_numbers(numbers):
```

```
    result = []
```

```
for number in numbers:  
    result.append(number * 2)  
  
return result
```

B1. Auxiliary Space complexity: $O(n)$

B2. Why does the memory usage grow with the input size?

The function creates a brand-new list result that stores one doubled value for every element in the input list. If the input has n elements, result must hold n elements too. Therefore the additional memory grows linearly with n , giving $O(n)$ auxiliary space.

Comparison Question

Compare Algorithm A and Algorithm B. Which one uses less auxiliary memory, and why?

Algorithm A uses less auxiliary memory. Algorithm A only needs a constant number of variables (total and the loop variable), so its auxiliary space is $O(1)$. Algorithm B must allocate a new list of size n proportional to the input, so its auxiliary space is $O(n)$. Therefore A is more memory-efficient than B.

Submission Check

Predicted before running: Yes

Tested in Python: Yes

Student signature: 20253801028